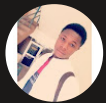


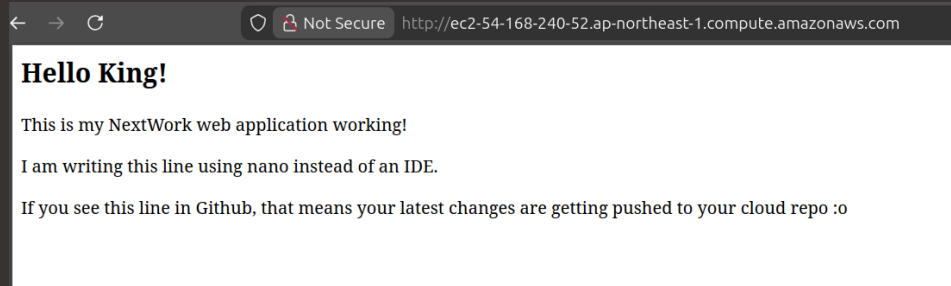


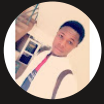
nextwork.org

Deploy a Web App with CodeDeploy



Ahmed Tetteh





Introducing Today's Project!

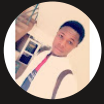
In this project, I will demonstrate how AWS CodeDeploy works in a CI/CD pipeline for deploying webapps. I'm doing this project to learn how to automate deployments using a combination of CodeDeploy and deployment scripts to see a live website at the end.

Key tools and concepts

Services I used were Amazon EC2, CodeArtifact, CodeBuild, CodeDeploy, S3, CloudFormation, IAM, CodeConnection, Github and VS Code. Key concepts I learnt includes deployment, deployment groups, deployment scripts(i.e. appspec.yml file), roll backs and the importance of rebuilding your build artifact before deployment.

Project reflection

This project took me approximately 4 hours(including error troubleshooting and background research). The most challenging part was combining the process flow from the development instance to the deployment instance. Also, I had to troubleshoot some errors I was getting during my deployment of the web app. It was most rewarding to see the successful deployment message and my live web app for the end users in on the deployment instance.

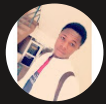


Ahmed Tetteh

NextWork Student

nextwork.org

This project is part five of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project in 2 days, i.e, 19th December.

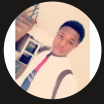


Deployment Environment

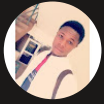
To set up for CodeDeploy, I launched an EC2 instance and VPC because I need to my web server to host the live webapp in a separate environment from my development environment. This provides me with the ability to make any changes in my development environment without it affecting my production environment or end users.

Instead of launching these resources manually, I used AWS CloudFormation to create my prod environment as a single unit or stack. When I need to delete these resources, I can easily delete them all together using a single delete command in the AWS CloudFormation console because, all those resources were created in a stack.

Other resources created in this template includes the Internet Gateway, Security Group, Subnet and Route Table. They're also in the template because, they contribute to creating an entirely separate web hosting environment that can be modified via the template, provide security and be replicated anytime. They also enable traffic control within that environment for our end-users or other resources such as databases.



Resources (11)					
Search resources					
Logical ID	Physical ID	Type	Status	Module	
DeployRoleProfile	NextWorkCodeDeployEC2Stack-DeployRoleProfile-6ASPQywXQheO	AWS::IAM::InstanceProfile	CREATE_COMPLETE		
InternetGateway	igw-0d1801c62149e083c	AWS::EC2::InternetGateway	CREATE_COMPLETE		
PublicInternetRoute	rtb-05d9ef65009168077j0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE		
PublicRouteTable	rtb-05d9ef65009168077	AWS::EC2::RouteTable	CREATE_COMPLETE		
PublicSecurityGroup	sg-003d543d335722d1c	AWS::EC2::SecurityGroup	CREATE_COMPLETE		
PublicSubnetA	subnet-022735f7824222506	AWS::EC2::Subnet	CREATE_COMPLETE		
PublicSubnetARouteTableAssociation	rtbassoc-0d6f05ef7f329a1ab	AWS::EC2::SubnetRouteTableAssociation	CREATE_COMPLETE		
ServerRole	NextWorkCodeDeployEC2Stack-ServerRole-NF4HrMQy41oR	AWS::IAM::Role	CREATE_COMPLETE		
VPC	vpc-03a99a40c00ddf13b	AWS::EC2::VPC	CREATE_COMPLETE		
VPCGatewayAttachment	IGWVpc-03a99a40c00ddf13b	AWS::EC2::VPCGatewayAttachment	CREATE_COMPLETE		
WebServer	i-0c0fbc3dec94c996a	AWS::EC2::Instance	CREATE_COMPLETE		



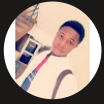
Deployment Scripts

Scripts are simply mini-programs that helps us automate the running of commands. To set up CodeDeploy, I also wrote scripts to automate deployment commands, i.e, these commands will my EC2 instance how to host my web app.

The 'install_dependencies.sh' will install the dependencies necessary to host my web app, such as Tomcat- the web server for Java web apps.

The start_server.sh will start both Apache(the web server) and Tomcat (the Java application server) and makes sure they start up automatically upon reboot.

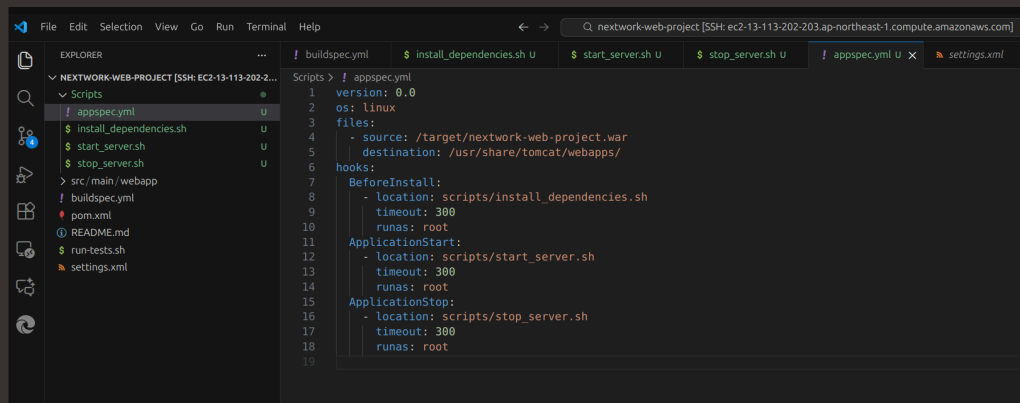
The stop_server.sh will ensure that our web services stop safely, if there are any processes running on them. This ensures that we can easily stop our web services by running a simple script, and properly identify the state of our web services.



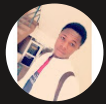
appspec.yml

Then, I wrote an appspec.yml file to provide instructions to CodeDeploy on how to deploy my web app, including the different phases of deployment. The key sections in appspec.yml are the Os type, files and deployment lifecycle events(specifying the exact moments or phases CodeDeploy should run commands)

I also updated buildspec.yml because CodeDeploy needs those instructions from our scripts and appspec.yml file deploy our java web app within our EC2 instance. So, all the instructions have been bundled into one java artifact CodeDeploy can take to deploy the web app within our hosting environment.



```
1 version: 0.0
2 os: linux
3 files:
4   - source: /target/nextwork-web-project.war
5     destination: /usr/share/tomcat/webapps/
6 hooks:
7   BeforeInstall:
8     - location: scripts/install_dependencies.sh
9       timeout: 300
10      runas: root
11   ApplicationStart:
12     - location: scripts/start_server.sh
13       timeout: 300
14      runas: root
15   ApplicationStop:
16     - location: scripts/stop_server.sh
17       timeout: 300
18      runas: root
19
```

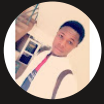


Setting Up CodeDeploy

A deployment group is simply a group of EC2 instances grouped together to deploy something. A CodeDeploy application is like a main folder for my deployment project. It contains all the different deployment configurations applications need for the deployment groups(i.e, can be a single EC2 instance)- can be deployment groups for testing, staging or production environment.

To set up a deployment group, you also need to create an IAM role for CodeDeploy to have the necessary permissions it needs for our deployment process. CodeDeploy needs to be able to access our EC2 instance we are going to use our web server, accesss our S3 bucket for the java artifact and more.

Tags are helpful for documenting our resource(i.e. we can easily identify what the resource is for) and flexibility, i.e. we can freely add new instances of the same tag to the deployment group for future deployments. I used the tag "role" to easily identify the purpose of my new EC2 instance from the CloudFormation template, i.e, webserver.



Environment configuration

Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment

☐ Amazon EC2 Auto Scaling groups

☒ Amazon EC2 instances

1 unique matched instance. [Click here for details](#)

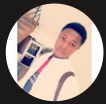
You can add up to three groups of tags for EC2 instances to this deployment group.
One tag group: Any instance identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.

Tag group 1

Key	Value - optional	
role	webserver	Remove tag

☐ On-premises instances

Matching instances
1 unique matched instance. [Click here for details](#)



Deployment configurations

Another key settings is the deployment configuration, which affects the rate at which an application is deployed or rolled out. I used `CodeDeployDefault.AllAtOnce`, so I can quickly deploy the web app on my single EC2 instance since am not working in a group of instances in a production environment.

In order to connect the deployment instance to CodeDeploy, a CodeDeploy Agent is also set up to serve as the intermediary between them by receiving the deployment instructions(the scripts,e.g, `appspec.yml`) from CodeDeploy and making sure they are run on the EC2 instance.

Agent configuration with AWS Systems Manager [Info](#)

**Complete the required prerequisites before AWS Systems Manager can install the CodeDeploy Agent.**
Make sure the AWS Systems Manager Agent is installed on all instances and attach the required IAM policies to them. [Learn more](#) 

Install AWS CodeDeploy Agent

☐ Never

☐ Only once

☒ Now and schedule updates

Basic scheduler

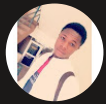
Cron expression

14

▼

Days

▼

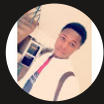


Success!

A CodeDeploy deployment represents a specific update to an application with its very own unique ID and history. The difference to a deployment group is that, a deployment group is like a settings file that contains all sorts of different configurations for an environment, while deployment is like a specific update made using that settings file.

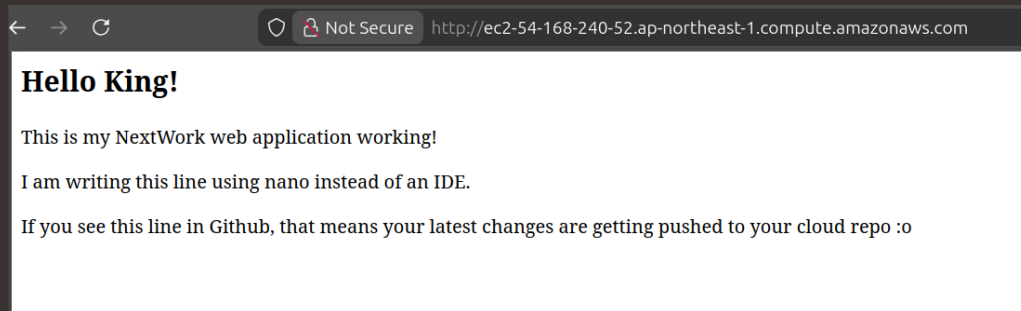
I had to configure a revision location, which means that CodeDeploy will fetch that build artifact (war or zipped file) from that location, i.e., in my case, an S3 bucket. So, it shows exactly where our deployable WAR file is stored.

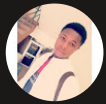
To check that the deployment was a success, I visited the Public IPV4 DNS in my EC2 console. I saw a live web app that's working and serving my end users!!!!!!!.



Ahmed Tetteh
NextWork Student

nextwork.org



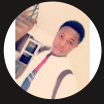


Disaster Recovery

In a project extension, I decided to include an error in my `stop_server` script to test out how the rollback features of CodeDeploy will affect my deployment. The intentional error I created was `"systemctl"`. This will cause the deployment to fail because when CodeDeploy runs the `stop_server` script first during the deployment, it will encounter the error saying `"systemctl: command not found"`, and exit out of deployment.

I also enabled rollbacks with this deployment, which means that CodeDeploy will allow you to roll back to the previous working version of your application before failure to minimize downtime.

When my deployment failed, the automatic rollback also failed to deploy because it couldn't revert back to a previously known stable deployment from the deployment history. CodeDeploy had to stop services and perform other actions in order to roll back to the stable old build artifact, but since our script was broken., it couldn't even run it to start the rollback. To actually recover from this deployment failure, I'd have to do a manual deployment recovery by fixing the script and redeploying a new version myself. In production environments, more sophisticated tools can be implement in the CI/CD pipeline, including AWS Codepipeline to perform automatic rollbacks.



Developer Tools > CodeDeploy > Deployments

Deployment history View details Actions Copy deployment Retry deployment

Q

	Deployment Id	Status	Deployment t...	Compute plat...	Application	Deployment g...	Revision locati...	Initiating event	Start time	End time
☐	d-IT4VB4BCG	Failed	In-place	EC2/On-premi...	nextwork-devo...	nextwork-devo...	s3://nextwork-...	CodeDeploy ro...	Dec 16, 2025 1...	Dec 16, 2025 1...
☐	d-B8L11PBCG	Failed	In-place	EC2/On-premi...	nextwork-devo...	nextwork-devo...	s3://nextwork-...	User action	Dec 16, 2025 1...	Dec 16, 2025 1...



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

